G-Computation: Intuition and Math

Davood Tofighi, Ph.D.

Overview

Welcome to Week 7. In our previous sessions, we explored how to control for confounding using methods that operate on the probability of treatment, namely Inverse Probability (IP) Weighting. This week, we turn to a powerful and conceptually distinct alternative: **G-Computation**.

Also known as the g-formula, standardization, or the substitution estimator, g-computation is one of the oldest and most intuitive methods for confounding adjustment. At its core, g-computation works by building a model of the world—specifically, a model of how the outcome depends on the treatment and confounders. It then uses this model to simulate a perfect randomized trial. We will ask our model: "What would the average outcome have been in our entire population if everyone had received the treatment?" and "What would it have been if, instead, no one had received the treatment?"

The difference between the answers to these two questions is our estimate of the average causal effect. This approach is powerful because it directly estimates the quantities of interest from the potential outcomes framework, $\mathbb{E}\left[Y^{a=1}\right]$ and $\mathbb{E}\left[Y^{a=0}\right]$. However, its validity rests heavily on a critical, untestable assumption: that our model of the world is correctly specified. We will dedicate significant time to understanding this trade-off and its practical implications for our research.

Learning Objectives

By the end of this week's module and lab, you will be able to:

1. **Describe** the three-step g-computation algorithm (model, predict, average) and explain how it simulates a target trial to control for confounding.
2. **State** and **interpret** the three key identification assumptions required for g-computation: conditional exchangeability, positivity, and consistency.
3. **Derive** the g-formula (standardization formula) and connect each of its mathematical terms to the practical steps of the g-computation algorithm.
4. **Implement** parametric g-computation in R by fitting an outcome model, generating counterfactual predictions, and averaging to estimate the Average Treatment Effect (ATE).

5. **Critically assess** the primary limitation of parametric g-computation—its reliance on correct model specification—and discuss its implications for causal inference.

R Packages

This week, we'll focus on outcome modeling and prediction. The `boot` package will be essential for calculating confidence intervals for our g-computation estimates.

```r
if (!require("pacman")) {
    install.packages("pacman")
}
pacman::p_load(
    tidyverse, # For data manipulation and visualization
    ggdag, # For plotting DAGs
    broom, # For tidying model outputs
    knitr, # For table formatting
    boot # For bootstrapping confidence intervals
)

# A consistent plot theme
theme_set(theme_minimal(base_size = 14))
```

1. Comprehensive Topic Explanation

The Logic of G-Computation (Standardization)

G-computation tackles confounding from the opposite direction of IP weighting. While weighting methods create a pseudo-population by modeling the *treatment assignment process* (`A ~ L`), g-computation works by modeling the *outcome generating process* (`Y ~ A + L`).

The method is best understood as a three-step algorithm that directly operationalizes the concept of standardization, a classic technique in epidemiology.

The Algorithm:

1. **Fit an Outcome Model:** First, using your observational data, fit a regression model for the outcome Y as a function of the treatment A and the full set of confounders L. This model, often called the "Q-model," learns the conditional expectation of the outcome, $\mathbb{E}[Y|A, L]$.
2. **Predict Potential Outcomes under Interventions:**

- **Intervention 1 (Everyone Treated):** Create a new dataset where the treatment variable A is set to 1 for *every individual* in your study. The confounder values L remain at their observed, real-world levels for each person. Use the model from Step 1 to predict the outcome for every person in this hypothetical dataset. These are the predicted potential outcomes under treatment, $\hat{Y}^{a=1}$.
- **Intervention 2 (No one Treated):** Create a second new dataset where A is set to 0 for everyone, again keeping their observed L values. Use the model from Step 1 to predict the outcome for everyone in this second hypothetical dataset. These are the predicted potential outcomes under no treatment, $\hat{Y}^{a=0}$.

3. **Average and Contrast:**
   - Calculate the average of all the predicted outcomes from Intervention 1. This is your estimate of the standardized mean outcome under treatment, $\hat{\mathbb{E}}\left[Y^{a=1}\right]$.
   - Calculate the average of all the predicted outcomes from Intervention 2. This is your estimate of the standardized mean outcome under no treatment, $\hat{\mathbb{E}}\left[Y^{a=0}\right]$.
   - The Average Treatment Effect (ATE) is the difference between these two means: ATE $= \hat{\mathbb{E}}\left[Y^{a=1}\right] - \hat{\mathbb{E}}\left[Y^{a=0}\right]$.

This procedure breaks the confounding link between L and A not by re-weighting, but by forcing the distribution of L to be identical in the two scenarios we compare (since both scenarios use the exact same population with their observed values of L).

The G-Formula: Mathematical Foundation

The algorithm described above is the computational implementation of the **g-formula**, or standardization formula. This formula provides the mathematical bridge from the unobservable world of potential outcomes to the observable world of data, under key assumptions.

The goal is to identify $\mathbb{E}[Y^a]$. The g-formula for a discrete set of confounders L is:

$$\mathbb{E}[Y^a] = \sum_l \mathbb{E}[Y|A = a, L = l] \times P(L = l)$$

For continuous confounders, the sum becomes an integral:

$$\mathbb{E}[Y^a] = \int \mathbb{E}[Y|A = a, L = l] f(l) dl$$

Connecting the Math to the Algorithm:

- $\mathbb{E}[Y|A = a, L = l]$: This is the conditional expectation of the outcome. In parametric g-computation, this is precisely what our regression model from **Step 1** aims to estimate. The model gives us a way to get this value for any combination of a and l.
- $\sum_l ... \times P(L = l)$: This part of the formula represents averaging over the entire population's distribution of confounders. In our algorithm, we accomplish this in **Steps 2 and 3**. By predicting the outcome for every single person in our original dataset (first under $A = 1$, then under $A = 0$), and then taking the mean of those predictions, we are effectively calculating the weighted average, where the weights are simply $1/N$ for each person, thus perfectly representing the empirical distribution of L in our sample.

Key Assumptions for Identification

For the g-formula to hold and for g-computation to yield a valid causal estimate, three core assumptions are required:

1. **Conditional Exchangeability (Ignorability):** $Y^a \perp\!\!\!\perp A|L$. We must have measured and adjusted for a set of covariates L that is sufficient to break all backdoor paths between treatment A and outcome Y. This is the crucial "no unmeasured confounding" assumption.
2. **Positivity (Overlap):** $P(A = a|L = l) > 0$ for all l with $P(L = l) \neq 0$. For every type of person in our population (i.e., for every level of L), there must be a non-zero chance of being both treated and untreated. If a certain type of person is *always* treated, we can never learn from the data what would have happened to them had they not been treated.
3. **Consistency:** An individual's observed outcome is their potential outcome corresponding to the treatment they actually received. This links the potential outcomes framework to the observed data.

> ⚠ The Achilles' Heel of G-Computation: Model Specification
>
> In addition to the three causal assumptions, parametric g-computation carries a fourth, statistical assumption: **correct model specification**. The entire method hinges on the Q-model ($Y \sim A + L$) being a correct representation of reality. If the true relationship between the confounders and the outcome is non-linear, but we fit a linear model, our predictions in Step 2 will be wrong, and our final ATE estimate will be biased due to **residual confounding**. This is the fundamental trade-off compared to IPW, which instead relies on a correctly specified *treatment* model.

2. Intuitive Clarifications of Challenging Ideas

Analogy: The Plant Scientist

Imagine a scientist wants to know the causal effect of a new fertilizer ($A = 1$) vs. a standard one ($A = 0$) on crop yield (Y). The scientist knows that soil quality (L) is a major confounder: better soil both improves yield directly and might influence which plots get the new, expensive fertilizer.

She has observational data from 100 different farm plots. A naive comparison of yield from plots that got the new fertilizer vs. the old one would be biased.

The G-Computation Approach:

1. **Model the Farm:** The scientist fits a regression model: `Yield ~ Fertilizer + Soil_Quality`. This model learns the fundamental relationship between these variables from the 100 observed plots.
2. **Simulate a Universe-Wide Experiment:**
   - **Scenario 1:** She takes her list of 100 plots, with their *actual observed soil quality*, and asks her model: "What would the yield have been for each of these 100 plots if they had *all* received the new fertilizer ($A = 1$)?".
   - **Scenario 2:** She does the same, but this time asks: "What would the yield have been for each of these 100 plots if they had *all* received the standard fertilizer ($A = 0$)?".
3. **Average the Results:** She calculates the average predicted yield across all 100 plots in Scenario 1 to get $\mathbb{E}\left[Y^{a=1}\right]$. She does the same for Scenario 2 to get $\mathbb{E}\left[Y^{a=0}\right]$. The difference is her causal effect estimate.

This works because she is comparing two hypothetical universes that are identical in every way (they have the same 100 plots with the same soil quality distribution), except for the fertilizer they received.

G-Computation vs. IP Weighting: A Comparison

These two methods are the foundational pillars of confounding adjustment. They approach the problem from opposite directions.

Feature	**G-Computation (Standardization)**	**IP Weighting (IPTW)**
Core Idea	Outcome Modeling & Simulation	Treatment Modeling & Re-weighting
Model Required	A model for the outcome: **`Y ~ A + L`** (the Q-model)	A model for the treatment: **`A ~ L`** (the propensity score model)

Feature	**G-Computation (Standardization)**	**IP Weighting (IPTW)**
Main Vulnerability	**Outcome model misspecification.** If the Q-model is wrong, the estimate is biased.	**Positivity violations.** If propensity scores are near 0 or 1, weights become extreme and estimates unstable.
Intuition	"What would the outcome look like in a standardized population?"	"What would the A-Y association look like in a pseudo-population where L no longer predicts A?"

3. Simple, Hand-Calculation Numerical Example(s)

Let's use a very simple dataset to perform g-computation by hand. We want to find the effect of a Tutoring program (A) on a final Exam Score (Y), adjusting for a binary confounder, Prior Grades (L).

Observed Data (8 Students):

Student	Prior Grades (L)	Tutoring (A)	Exam Score (Y)
1	Good	1	95
2	Good	1	90
3	Good	0	88
4	Good	0	85
5	Weak	1	80
6	Weak	1	78
7	Weak	0	70
8	Weak	0	68

Step 1: Fit an Outcome Model. Let's fit a simple linear model to our data. Let `L_good` be an indicator variable (1 if "Good", 0 if "Weak"). Suppose our model fit is:

$$\widehat{Score} = 69 + 11 \times A + 16 \times L_{good}$$

- **Interpretation:** The baseline score for a weak student without tutoring is 69. Tutoring adds 11 points, and having good prior grades adds 16 points.

Step 2: Predict Potential Outcomes for the Entire Population.

Scenario 1: Everyone gets Tutoring ($A = 1$) We use our model to predict the score for all 8 students, forcing $A = 1$.

Student	Prior Grades (L_{good})	Hypothetical A	Predicted Score $\hat{Y}^{a=1}$
1	1	1	$69 + 11(1) + 16(1) = 96$
2	1	1	$69 + 11(1) + 16(1) = 96$
3	1	1	$69 + 11(1) + 16(1) = 96$
4	1	1	$69 + 11(1) + 16(1) = 96$
5	0	1	$69 + 11(1) + 16(0) = 80$
6	0	1	$69 + 11(1) + 16(0) = 80$
7	0	1	$69 + 11(1) + 16(0) = 80$
8	0	1	$69 + 11(1) + 16(0) = 80$

Scenario 2: No one gets Tutoring ($A = 0$) Now we predict for all 8 students, forcing $A = 0$.

Student	Prior Grades (L_{good})	Hypothetical A	Predicted Score $\hat{Y}^{a=0}$
1	1	0	$69 + 11(0) + 16(1) = 85$
2	1	0	$69 + 11(0) + 16(1) = 85$
3	1	0	$69 + 11(0) + 16(1) = 85$
4	1	0	$69 + 11(0) + 16(1) = 85$
5	0	0	$69 + 11(0) + 16(0) = 69$
6	0	0	$69 + 11(0) + 16(0) = 69$
7	0	0	$69 + 11(0) + 16(0) = 69$
8	0	0	$69 + 11(0) + 16(0) = 69$

Step 3: Average and Contrast.

- $\hat{\mathbb{E}}\left[Y^{a=1}\right] = \frac{96+96+96+96+80+80+80+80}{8} = \frac{4\times96+4\times80}{8} = 88$
- $\hat{\mathbb{E}}\left[Y^{a=0}\right] = \frac{85+85+85+85+69+69+69+69}{8} = \frac{4\times85+4\times69}{8} = 77$
- $\text{ATE} = 88 - 77 = 11$

Conclusion: Our g-computation estimate of the causal effect of tutoring is an 11-point increase in exam scores. This correctly matches the coefficient for `A` in our outcome model, because the model was linear and had no interactions.

4. R Code Examples & Interpretation

Let's implement the g-computation algorithm in R and, crucially, use the bootstrap method to obtain a confidence interval.

Step 1: Simulate Confounded Data

```r
set.seed(42)
n <- 1000
# L is a Continuous Confounder (e.g., age)
L <- rnorm(n, mean = 50, sd = 10)
# Treatment A Depends on L (confounding)
prob_A <- plogis(-5 + 0.1 * L)
A <- rbinom(n, 1, prob_A)
# Outcome Y Depends on A and L. True ATE = -15
Y <- 100 - 15 * A + 2 * L + rnorm(n, 0, 8)

df <- tibble(Y, A, L)

# Naive (biased) Analysis for Comparison
naive_model <- lm(Y ~ A, data = df)
tidy(naive_model) ▷
    filter(term == "A") ▷
    kable(digits = 2)
```

term	estimate	std.error	statistic	p.value
A	1.14	1.26	0.9	0.37

Interpretation: The naive analysis shows an effect of 1.14 on the outcome, which is biased towards the null compared to the true effect of -15. This is because older people (higher L) are more likely to get the treatment and also tend to have higher outcomes, making the treatment look less effective than it truly is.

Step 2: Implement G-Computation with Bootstrapped Confidence Intervals

The best way to get a confidence interval for a g-computation estimate is via the bootstrap. This involves repeatedly resampling our data and running the entire algorithm on each resample to see how much the ATE estimate varies.

```r
# First, Write a Function that Performs the G-computation Steps.
# This Function Will Be Called by the boot() Function.
```

```r
calculate_ate <- function(data, indices) {
    # Allow Boot to Select Resampled Datasets
    d <- data[indices, ]

    # 1. Fit the Outcome Model on the Resampled Data
    q_model <- lm(Y ~ A + L, data = d)

    # 2. Create Counterfactual Datasets
    d_a1 <- d ▷ mutate(A = 1) # Everyone treated
    d_a0 <- d ▷ mutate(A = 0) # No one treated

    # 3. Predict Potential Outcomes
    y_hat_a1 <- predict(q_model, newdata = d_a1)
    y_hat_a0 <- predict(q_model, newdata = d_a0)

    # 4. Average and Contrast
    mean_y1 <- mean(y_hat_a1)
    mean_y0 <- mean(y_hat_a0)

    return(mean_y1 - mean_y0)
}

# Now, Run the Bootstrap
set.seed(123)
bootstrap_results <- boot(
    data = df,
    statistic = calculate_ate,
    R = 999 # Number of bootstrap replications
)

# View the Results
print(bootstrap_results)
```

```
ORDINARY NONPARAMETRIC BOOTSTRAP


Call:
boot(data = df, statistic = calculate_ate, R = 999)
```

```
Bootstrap Statistics :
    original       bias    std. error
t1* -15.2348 -0.009824715   0.5744005
```

```r
# Calculate a 95% Percentile Confidence Interval
ci_boot_gcomp <- boot.ci(bootstrap_results, type = "perc")
ci_boot_gcomp
```

```
BOOTSTRAP CONFIDENCE INTERVAL CALCULATIONS
Based on 999 bootstrap replicates

CALL :
boot.ci(boot.out = bootstrap_results, type = "perc")

Intervals :
Level     Percentile
95%   (-16.40, -14.08 )
Calculations and Intervals on Original Scale
```

Interpretation:

- **`original`**: This value (`t1*`) is the ATE calculated on our original, un-resampled dataset. It should be very close to the true effect of -15.
- **`bias`**: This is the difference between the average of the bootstrap estimates and the original estimate. We want this to be small.
- **`std. error`**: This is the standard deviation of the bootstrap estimates, which is our bootstrap standard error for the ATE.
- **`boot.ci` output**: This provides the 95% confidence interval. A correct interpretation would be: "Using parametric g-computation, the estimated average causal effect of the treatment is -15.23. We are 95% confident that the true average causal effect lies between -16.4 and -14.08, based on our bootstrap. Since this interval is far from 0, we can conclude there is a statistically significant, negative causal effect.

5. Hands-On R Lab

Lab: Estimating the Effect of a Diet Plan on Cholesterol

Purpose: This lab will provide hands-on experience implementing the full parametric g-computation workflow, including fitting an outcome model that includes interactions and using the bootstrap to generate confidence intervals.

Lab Objectives:

1. Fit a parametric outcome model (Q-model) to observational data.
2. Implement the g-computation algorithm to estimate the ATE.
3. Write a function to perform g-computation that is compatible with the `boot` package.
4. Generate and interpret bootstrapped confidence intervals for the ATE.

Scenario: You are a health analyst examining the effect of a new, intensive diet plan ($A = 1$) versus standard advice ($A = 0$) on cholesterol reduction (Y, measured in mg/dL). You have observational data and suspect that **baseline BMI** (L) is a major confounder, as individuals with higher BMI might be more likely to be assigned to the intensive plan and also have different cholesterol outcomes. Furthermore, you suspect the diet plan might be *more effective* for people with higher BMI (i.e., there is effect modification).

Task 1: Simulate Data and Examine the Naive Effect

Instructions:

1. Generate the confounded data using the block below.
2. Run a naive linear model (`lm(Y ~ A, ...)`) to see the biased association.
3. The true ATE in this simulation is -10. How does the naive estimate compare?

```r
set.seed(123)
n_lab <- 500
# L = Baseline BMI (confounder)
L <- rnorm(n_lab, 28, 4)
# A = Assigned to Intensive Diet Plan
prob_A <- plogis(-3 + 0.1 * L)
A <- rbinom(n_lab, 1, prob_A)
# Y = Cholesterol Reduction. Note the Interaction Term A*L (effect modification)
Y <- 5 - 10 * A + 0.5 * L - 0.2 * A * L + rnorm(n_lab, 0, 5)
lab_df <- tibble(Y, A, L)
```

Solution to Task 1

```
naive_lab_model <- lm(Y ~ A, data = lab_df)
tidy(naive_lab_model) ▷ filter(term == "A")
```

term	estimate	std.error	statistic	p.value
A	-13.6413	0.4737788	-28.79256	0

Interpretation: The naive estimate is around -5.5, which is severely biased towards the null compared to the true ATE of -10. This is because people with high BMI (who are more likely to get the diet plan) have smaller cholesterol reductions in this scenario, making the plan look less effective than it is.

Task 2: Fit the Outcome Model (Q-model)

Instructions: Based on the data-generating process, we know there is an interaction between the diet plan `A` and baseline BMI `L`. Fit a linear model for the outcome `Y` that includes `A`, `L`, and their interaction term (`A * L` or `A:L`). Examine the coefficients.

```
# TODO: Fit the Q-model with the Interaction Term
q_model_lab <- lm( ... )

# TODO: Tidy the Model and Look at the Coefficients
tidy( ... )
```

Solution to Task 2

```
q_model_lab <- lm(Y ~ A * L, data = lab_df)
tidy(q_model_lab)
```

term	estimate	std.error	statistic	p.value
(Intercept)	4.8615234	2.2274227	2.182578	0.0295352
A	-7.2557083	3.4518728	-2.101963	0.0360602
L	0.4892053	0.0808849	6.048169	0.0000000
A:L	-0.2509521	0.1206342	-2.080272	0.0380133

Interpretation: The model has successfully recovered coefficients that are close to the true parameters in our simulation (`Intercept=5`, `A=-10`, `L=0.5`, `A:L=-0.2`). This correctly specified Q-model is the foundation for our g-computation.

Task 3: Implement the G-Computation Bootstrap

Instructions: Following the lecture example, write a function `calculate_ate_diet` that takes `data` and `indices` as input. Inside the function, you should:

1. Fit the correct Q-model (`Y ~ A * L`).
2. Create the two counterfactual datasets.
3. Predict the outcomes for both.
4. Return the difference in the mean predictions.

Then, call the `boot()` function using your new function to get the bootstrapped results and a 95% confidence interval.

```
# TODO: Write the G-computation Function
calculate_ate_diet <- function(data, indices) {
    d <- data[indices, ]

    # Step 1: Fit Model
    q_model <- lm( ... )

    # Step 2: Create Counterfactual Data
    d_a1 <- ...
    d_a0 <- ...

    # Step 3: Predict
    y1_hat <- ...
    y0_hat <- ...

    # Step 4: Average and Contrast
    ...
    return( ... )
}

# TODO: Call boot() with Your Function
boot_results_lab <- boot( ... )

# TODO: Print the Results and the Confidence Interval
print( ... )
boot.ci( ... )
```

Solution to Task 3

```r
calculate_ate_diet <- function(data, indices) {
    d <- data[indices, ]

    # Step 1: Fit Model
    q_model <- lm(Y ~ A * L, data = d)

    # Step 2: Create Counterfactual Data
    d_a1 <- d |> mutate(A = 1)
    d_a0 <- d |> mutate(A = 0)

    # Step 3: Predict
    y1_hat <- predict(q_model, newdata = d_a1)
    y0_hat <- predict(q_model, newdata = d_a0)

    # Step 4: Average and Contrast
    mean_y1 <- mean(y1_hat)
    mean_y0 <- mean(y0_hat)

    return(mean_y1 - mean_y0)
}

set.seed(321)
boot_results_lab <- boot(
    data = lab_df,
    statistic = calculate_ate_diet,
    R = 999
)

print(boot_results_lab)
```

```
ORDINARY NONPARAMETRIC BOOTSTRAP


Call:
boot(data = lab_df, statistic = calculate_ate_diet, R = 999)


Bootstrap Statistics :
```

```
     original      bias    std. error
t1* -14.31709 -0.00969678   0.4702048
```

```r
boot.ci(boot_results_lab, type = "perc")
```

```
BOOTSTRAP CONFIDENCE INTERVAL CALCULATIONS
Based on 999 bootstrap replicates

CALL :
boot.ci(boot.out = boot_results_lab, type = "perc")

Intervals :
Level     Percentile
95%   (-15.23, -13.40 )
Calculations and Intervals on Original Scale
```

Task 4: Final Reflection and Interpretation

1. **Interpretation:** Look at the `original` estimate from your bootstrap output. How close is it to the true ATE of -10? Write a one-sentence conclusion for a collaborator, reporting your point estimate and 95% confidence interval.
2. **Conceptual:** Why did we have to include an interaction term (`A*L`) in our Q-model for this lab, but not in the main lecture example? What would have happened to our ATE estimate if we had forgotten it?
3. **Retrieval Quiz:**
   - What are the three main steps of the g-computation algorithm?
   - What is the name of the key untestable assumption that states we have measured all common causes of the treatment and outcome?
   - G-computation is to the **outcome model** as IP Weighting is to the _____ **model**.

6. Common Pitfalls & Checks

> ⚠ The Peril of Model Misspecification
>
> The single greatest risk in parametric g-computation is that your outcome model is wrong. If you omit important confounders, forget to model non-linearities (e.g., using `L + I(L^2)`), or miss key interaction terms, your Q-model will be misspecified. This means your predictions of potential outcomes will be biased, and this bias will be directly inherited by your final ATE